FICHE PROJET – SÉCURITÉ INFORMATIQUE

LIVRABLE 1

Conception & Cahier des charges

Module	Sécurité informatique
Parcours	Licence Génie Logiciel et Systèmes d'Information
Niveau	3ème année – Semestre 1
Modalité	Individuel
Application	SecureBlog – Mini-blog avec authentification et commentaires
Dépôt	Espace de soumission sur Kedux

1. Présentation de l'application choisie

SecureBlog est une application web de type mini-blog permettant aux utilisateurs de s'inscrire, se connecter, publier des articles et commenter les publications des autres membres.

L'application a été conçue pour être volontairement simple afin de se concentrer pleinement sur les aspects de sécurité, tout en respectant l'ensemble des exigences du cahier des charges du projet :

- Un système d'authentification complet (inscription + connexion).
- Au moins une fonctionnalité de saisie et d'affichage dynamique (commentaires).
- Une base de données relationnelle pour le stockage des informations.

Elle servira de support pédagogique pour démontrer puis corriger les vulnérabilités suivantes (conformément au Top 10 OWASP) :

- Injection SQL (SQLi) : contournement d'authentification ou extraction de données.
- Cross-Site Scripting (XSS) : injection de scripts malveillants (Stored / Reflected).
- Gestion défaillante des sessions / mots de passe : absence de hachage, cookies vulnérables.
- (Bonus) Broken Access Control (IDOR) : accès non autorisé aux ressources d'un autre utilisateur.

2. Architecture globale

2.1 Stack technique

Couche	Technologie	Justification
Frontend	HTML5 + CSS3 + JavaScript	Simple, sans framework lourd, facile à analyser
Backend	PHP 8 (procédural / léger)	Idéal pour démontrer SQLi et XSS de manière claire
Base de données	MySQL / MariaDB ou SQLite	Base relationnelle obligatoire du projet

Serveur local	XAMPP / WAMP / Laragon	Environnement de développement et de test rapide

2.2 Architecture en 3 tiers

Client (Navigateur) ↔ HTTP ↔ Serveur Web (Apache + PHP) ↔ Base de données (MySQL)

2.3 Pages principales de l'application

- index.php — Liste des articles publiés
- register.php / login.php — Inscription et connexion des utilisateurs
- post.php — Affichage d'un article et de ses commentaires
- new_post.php — Création d'un nouvel article (utilisateur connecté)
- profile.php — Page de profil utilisateur
- logout.php — Déconnexion et destruction de session

3. Modélisation de la base de données

La base de données relationnelle est composée de trois tables principales liées entre elles.

3.1 Table users

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique de l'utilisateur
username	VARCHAR(50)	UNIQUE, NOT NULL	Nom d'utilisateur
email	VARCHAR(100)	UNIQUE, NOT NULL	Adresse e-mail
password	VARCHAR(255)	NOT NULL	Mot de passe (clair en V1, hashé en V2)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date d'inscription

3.2 Table posts

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique de l'article
user_id	INT	FK → users.id	Auteur de l'article
title	VARCHAR(200)	NOT NULL	Titre de l'article
content	TEXT	NOT NULL	Contenu de l'article
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date de publication

3.3 Table comments

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique du commentaire
post_id	INT	FK → posts.id	Article concerné
user_id	INT	FK → users.id	Auteur du commentaire
content	TEXT	NOT NULL	Contenu du commentaire
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date de publication

3.4 Relations

- Un utilisateur peut avoir plusieurs articles (relation 1-N).
- Un article peut avoir plusieurs commentaires (relation 1-N).
- Un utilisateur peut écrire plusieurs commentaires (relation 1-N).

4. Identification des fonctionnalités exposées aux risques de sécurité

Le tableau ci-dessous identifie les fonctionnalités de l'application les plus exposées aux vulnérabilités et précise le type de risque associé à chacune d'elles.

Fonctionnalité	Vulnérabilité ciblée	Description du risque
Connexion / Inscription	Injection SQL + Gestion des mots de passe	Requête SQL construite par concaténation → contournement d'authentification. Mots de passe stockés en clair.
Affichage des commentaires	XSS Stored	Le contenu du commentaire est affiché sans échappement → injection de scripts malveillants persistants.
Recherche d'articles (optionnel)	XSS Reflected / SQLi	Paramètre GET non filtré pouvant être exploité pour une injection.
Page de profil / Voir un post	Broken Access Control (IDOR) – Bonus	Accès possible à un profil ou post d'un autre utilisateur via modification de l'ID dans l'URL.
Gestion de session	Session Management	Cookie de session prévisible, absence de HttpOnly / Secure, durée de vie trop longue, pas de régénération d'ID.

5. Objectifs de sécurité du projet

5.1 Version 1 – Intentionnellement vulnérable

- Injection SQL sur le formulaire de connexion (contournement d'authentification).
- XSS Stored sur les commentaires (scripts persistants).
- Mots de passe stockés en clair dans la base de données.
- Cookies de session faibles (pas de flags de sécurité, ID prévisible).

5.2 Version 2 – Sécurisée et corrigée

- Utilisation de requêtes préparées (PDO ou mysqli) pour neutraliser les SQLi.
- Échappement / sanitisation des sorties avec htmlspecialchars() pour neutraliser les XSS.
- Hachage fort des mots de passe avec password_hash() et password_verify().
- Cookies de session sécurisés (HttpOnly, Secure, SameSite).
- Contrôle d'accès strict (vérification de propriété des ressources).

Ce document constitue le premier livrable du projet. Il servira de base à la conception des deux versions de l'application (vulnérable puis sécurisée) ainsi qu'à la démonstration des Proof of Concept lors de la soutenance.